COMP3320/6464 Week 5, Lecture 1

Rhys Hawkins

Semester 2, 2024

Overview

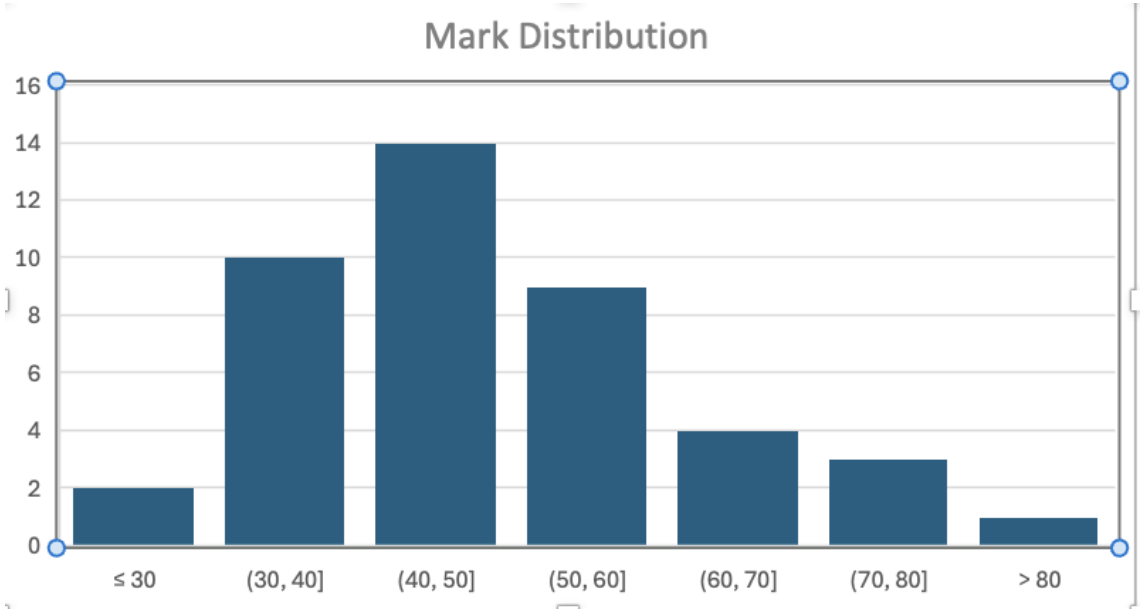
Today

- Mid-semester Test Review

This week

- SIMD
- SIMD intrinsics

Preliminary Mid-semester Test Results



- Range 3 .. 83
- Median 47
- For every question, someone got full marks

Preliminary Mid-semester Test Results

- Meeting with tutors this week
- **Raw** Marks and feedback on Wattle by end of week

Review: Basic Optimization

```
float compute_series_sum(float x, int N, float coeff[])
{
  double sum = 0.0;
  for (int i = 0; i <= N; i ++) {
    sum += coeff[i] / pow(x, i);
  }
  return sum;
}
```

Review: Basic Optimization: Step 1

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum = 0.0;
  for (int i = 0; i <= N; i ++) {
    sum += coeff[i] / pow(x, i);
  }
  return sum;
}
```

- Remove conversions between double/float

Review: Basic Optimization: Step 2

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum = 0.0;
  float denominator = 1.0;
  for (int i = 0; i <= N; i ++) {
    sum += coeff[i] / denominator; /* pow(x, i) */
    denominator *= x;
  }
  return sum;
}
```

- Remove pow
- pow only depends on i, so it can be computed in loop

Review: Basic Optimization: Step 3

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum = 0.0;
  float rx = 1.0/x;
  float denominator = 1.0;
  for (int i = 0; i <= N; i ++) {
    sum += coeff[i] * denominator; /* pow(x, i) */
    denominator *= rx;
  }
  return sum;
}
```

- Remove divisions from within loop
- Recall division is about 15 cycles compared to 4 for multiply.

Review: Basic Optimization: Loop unroll

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum = 0.0;
  float denominator = 1.0;
  float rx = 1.0/x;
  for (int i = 0; i <= N; i += 2) {
    sum += coeff[i] * denominator;
    denominator *= rx;
    sum += coeff[i + 1] * denominator;
    denominator *= rx;
  }
  return sum;
}
```

- In this form, it is quite easy to see that each statement in the loop depends on the previous so loop unrolling would not improve performance significantly.
- We need to introduct auxiliary variables.

Review: Basic Optimization: Loop unroll

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum1 = 0.0, sum2 = 0.0;
  float denominator = 1.0;
  float rx = 1.0/x;
  for (int i = 0; i <= N; i += 2) {
    sum1 += coeff[i] * denominator;
    denominator *= rx;
    sum2 += coeff[i + 1] * denominator;
    denominator *= rx;
  }
  return sum1 + sum2;
}
```

- We can eliminate the dependency between sums, but there remains a dependency caused by the denominator variable.

Review: Basic Optimization: Loop unroll

```
float compute_series_sum(float x, int N, float coeff[])
{
  float sum1 = 0.0, sum2 = 0.0;
  float rx = 1.0/x;
  float rx2 = rx*rx;
  float denominator1 = 1.0;
  float denominator2 = rx;
  for (int i = 0; i <= N; i += 2) {
    sum1 += coeff[i] * denominator1;
    sum2 += coeff[i + 1] * denominator2;
    denominator1 *= rx2;
    denominator2 *= rx2;
  }
  return sum1 + sum2;
}
```

- Loop unroll now has two independent iterations.
- In terms of superscalar architecture, the updates to sumx can be initiated at the same time on separate ports (likewise with the denominatorx updates).
- To take better advantage of pipelining, we could unroll further.

Review: Cache Read Miss Rate

```
struct particle {
  float x, y, z;
  double mass;
}
```

- You're given sizeof of float, double and struct particle and told that the compiler has padded the structure to 24 bytes.
- Within the loop, each member of the structure is read.
- The key is how 24 bytes interacts with a 32 byte cache line

Review: Cache Read Miss Rate: Layout in memory

x0	y0	z0	m0	m0	pad	x1	y1
z1	m1	m1	xx	x2	y2	z2	m2
m2	pad	x3	y3	z3	m3	m3	pad
x4	y4	z4	m4	m4	pad	x5	y5

- Using abbreviations: x0 $\equiv$ particles[0].x
- Each cell above is 4 bytes (our smallest type is float which is 4 bytes).
- We've arranged into 8 columns which gives 32 bytes per row which is the same as our cache line size.
- Pattern repeats every 32 $\times$ 3 bytes

Review: Cache Read Miss Rate: Layout in memory

Cache Line								
1	x0	y0	z0	m0	m0	pad	x1	y1
2	z1	m1	m1	pad	x2	y2	z2	m2
3	m2	pad	x3	y3	z3	m3	m3	pad
4	x4	y4	z4	m4	m4	pad	x5	y5

- We can then assume that the cache lines could be aligned as above
- So loading particles[0].x moves x, y, z, mass of the particle[0] and x and y of particle[1] into cache.

Review: Cache Read Miss Rate: Layout in memory

Cache Line								
1	x0	y0	z0	m0	m0	pad	x1	y1
2	z1	m1	m1	pad	x2	y2	z2	m2
3	m2	pad	x3	y3	z3	m3	m3	pad

- Red indicates cache misses
- 3 Cache misses
- 13 Cache hits
- Cache miss ratio is cache misses over cache hits plus cache misses
- 3/(13 + 3) = 3/16

Review: Cache Miss Rate

- We will cover more complex examples in week 6.
- Understanding cache read/write hit/miss rates is often critical for performance.

Review: Cache Associativity

- Many people didn't answer the Cache Associativity question.
- Let's review cache associativity from a low level look at the operation of cache.
- As an aid to understanding cache thrashing.
- And more generally cache operation.

Review: Cache Associativity

- What happens when read A[i]? What does the address of A[i] look like?

```
float A[5];
printf("%p\n", &A[0]);
printf("%p\n", &A[1]);
```

- The output is

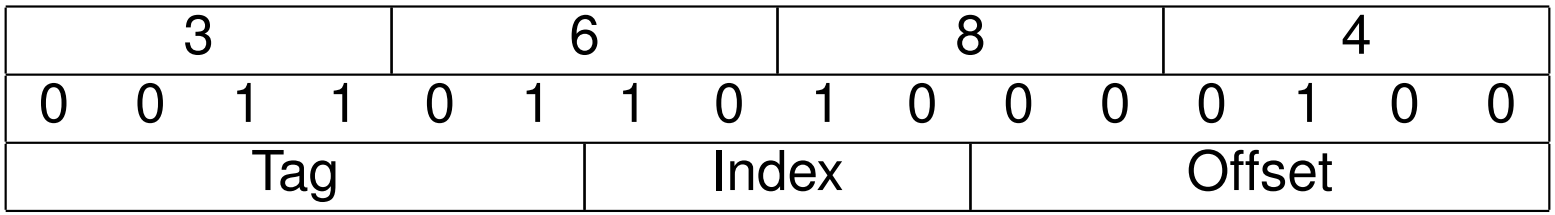
```
0x16ce93684
0x16ce93688
```

- What does this tell you?
- `sizeof(float) = 4` so the minimum addressable unit is 1 byte

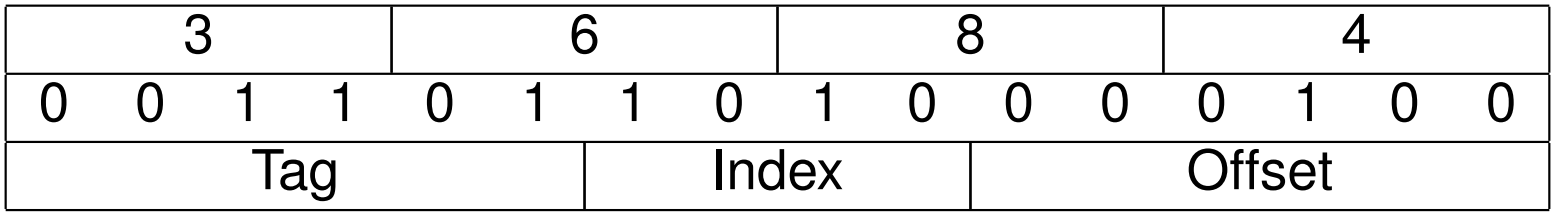
Review: Cache Associativity

An example, similar to a question on the mid-semester test

- Given a cache line size of 64 bytes, we need 6 bits to address every byte in a cache line (because $2^6 = 64$).
- If we have 1 kB of Direct Mapped Cache
  - 1024 bytes/64 bytes = 16 cachelines
  - To address every cache line we need 4 bits (because $2^4 = 16$)
- Imagine we are using 16 bit addresses, then every address in memory can be considered as follows (taking 0x3684 as an example)



Review: Cache Associativity

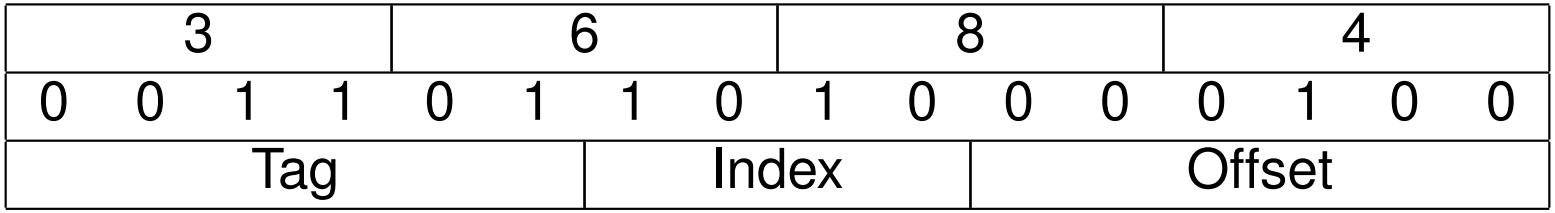


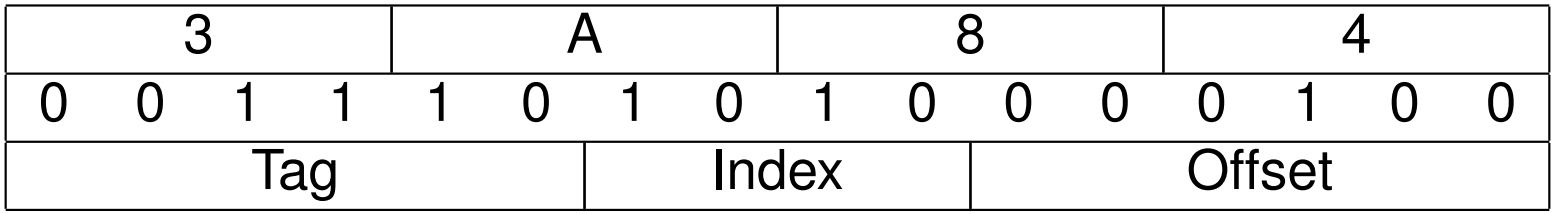
Mapping from memory address to Cache

- Tag is a unique identifier of the addresses that can be used to identify which address is currently in the cache line.
- Index determines which cache line the address goes to.
- Offset is the offset of a byte in the cache line
- How does a hit/miss work?

Review: Cache Associativity

To understand cache thrashing (at a low level), consider two addresses 0x3684 and 0x3A84





- Index is the same for both addresses, so these addresses a both mapped to the same cache line (10th line as index = 1010_b = 10)
- In general, if two addresses are 10000000000_b or $0x400$ or 2^{10} or 1024 bytes apart, they will store in the same cache line.
- In a direct mapped cache, it is no accident that this happens to be the same size as the cache we've used in this example.

Review: Cache Associativity

So what happens when we have code like this

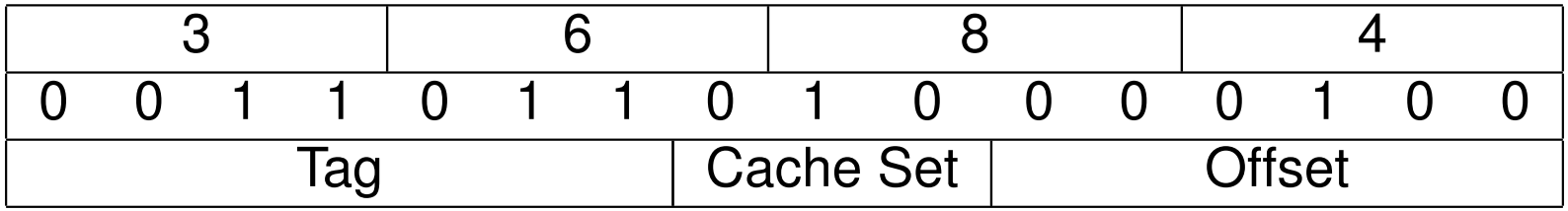
```
float A[N], B[N];
float s = 0.0;
for (int i = 0; i < N; i ++) {
  s += A[i] * B[i];
}
```

- If the offset between A[0] and B[0] is 1024 bytes, then there will be cache thrashing, ie load A[0] (miss), load B[0] (miss) evicts A[0..7], load A[1] (miss) evicts B[0..7], etc
- If the offset between A[0] and B[0] is an integer multiple of 1024 bytes, then the same thing will occur.
- With `sizeof(float)` = 4, this means that for N = 256, 512, 768, etc we will have cache thrashing
- Note that for values near 256 etc, we will have some level of “cache thrashing” due to cache line overlaps

Review: Cache Associativity

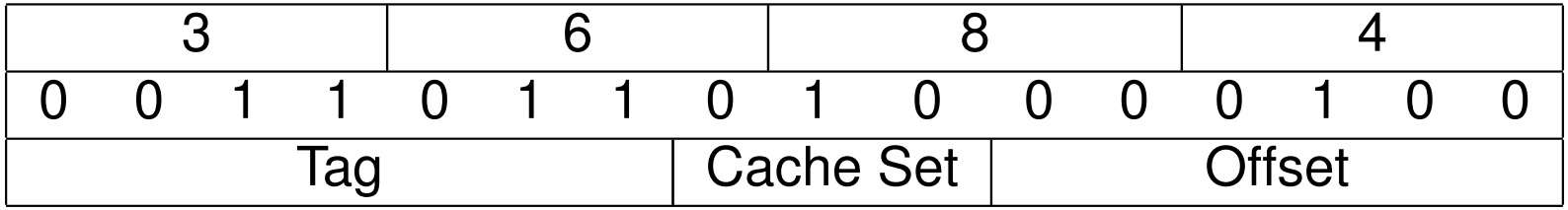
Now, what about cache associativity? Take the system from the question in the test where cache size was 1024 bytes, cache line size is 64 bytes and we have 2 way associativity.

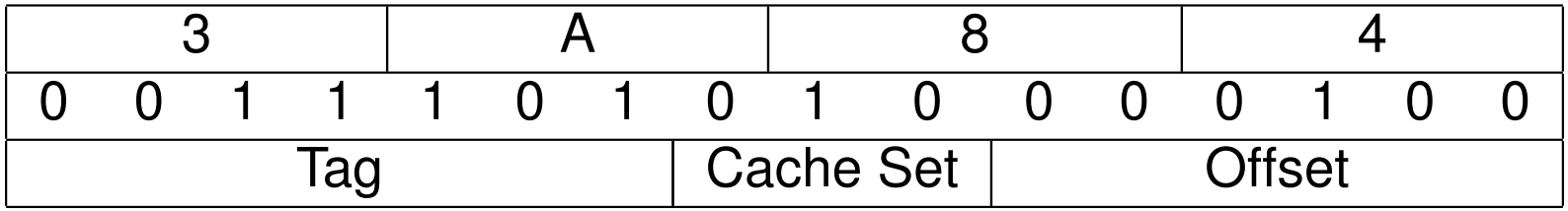
- Now instead of mapping an address to a distinct cache line in the cache (direct mapping), each address is mapped to a “cache set”.
- There are 16 cache lines of 64 bytes to give 1024 byte cache.
- Therefore there are 8 cache sets with 2 cache lines per set.
- Now an address gets mapped to a cache set instead.
- There are still 6 bits of the address used for the offset into the cache line
- But there are only 3 bits (because $2^3 = 8$) required to address the cache set to use



Review: Cache Associativity

To understand cache thrashing consider again two addresses 0x3684 and 0x3A84





- Note again that cache set is the same for both addresses. But the benefit here is that our cache set can hold two cache lines at a time given two way associativity.
- Here the offset is now 2^9 or 512 bytes (note that 2-way associativity has reduced this offset by 2, again no accident)
- But what happens when we have three reads?

Review: Cache Associativity

```
float A[N], B[N], C[N];
float s = 0.0;
for (int i = 0; i < N; i ++) {
  s += A[i] + B[i] * C[i];
}
```

- Cache sets use a LRU replacement policy to determine which slot in a set to use (and which cache line to evict).
- If B[0] is offset 512 bytes from A[0] and C[0] is offset 512 bytes from B[0], then the following sequence will occur
  1. Load A[0] (miss) A[0..7] stored in first slot of cache set
  2. Load B[0] (miss) B[0..7] stored in second slot
  3. Load C[0] (miss) C[0..7] stored in first slot, A[0..7] evicted
  4. Load A[1] (miss) A[0..7] stored in second slot, B[0..7] evicted
  5. Load B[1] (miss) B[0..7] stored in first slot, C[0..7] evicted
  6. and so on
- So N = 128 will cause cache thrashing in this case
- It can be easily obtained by 1024/cache associativity/sizeof(float) = 1024/2/4 = 128
- Again a positive integer multiple of 128 also causes thrashing.
- And again, numbers close to 128 still have some degree of thrashing due to cache line size

Summary

- **Raw** marks on Wattle this week
- The mark for the Mid-Semester may need to be scaled, but this will happen in the moderation of the course which happens after all marks are in.
- SIMD in remaining Lectures and Labs this week.